GitHub REST API Data Collection and Cleaning

Purpose

This module retrieves and processes repository metadata from the GitHub REST API for repositories created between 2007 and 2024.
The collected data serves as the basis for long-term programming language trend analysis in terms of code volume and repository activity.

Key Features

- Authenticated REST API access using a GitHub token
- Annual collection of repository metadata from 2007 to 2024
- Extraction of:
  - Repository name
  - Creation date
  - Star count
  - Language breakdown (via separate language API call)
- Language normalization and filtering using a defined whitelist
- Aggregation and export of all valid entries to a unified JSON dataset

---

Output Structure

All data is stored under the `./RestAPIData/` directory.

- `Rest_API_Request<year>.json` : Raw data per year (if collected)
- `All_API_Requests_Cleaned.json` : Final cleaned and validated dataset
- `api_languages_unique.csv` : Unique languages used across all repositories
- `api_blacklist.csv` : Languages excluded based on predefined criteria

Processing Steps

1. **API Request and Rate Handling**
   - GitHub repositories are queried using the `/search/repositories` endpoint for each target year.
   - A second request per repository fetches the language usage breakdown.
   - A delay (`time.sleep`) is applied between requests to respect rate limits.
2. **Filtering and Validation**

- Ensures all required fields are present (`full_name` , `created_at` , `languages` , `stargazers_count`)
- Duplicate repositories are removed based on `full_name`
- Only valid numerical star counts and non-empty language dictionaries are retained

3. **Language Normalization**
   - Common variants are mapped to standardized names (e.g. `Javascript` → `JavaScript` , `Golang` → `Go`)
   - Non-programmatic or invalid entries are removed using a predefined whitelist
4. **Final Cleaning**
   - Records with missing or invalid fields are excluded
   - Only repositories with at least one valid programming language are retained

---

Inclusion Criteria for Languages

A language is included if it:

- Supports control structures (`if` , `for` , `while` , etc.)
- Is general-purpose and Turing-complete or near-complete
- Is not limited to markup, query, or configuration roles

Examples:

- Included: Python, JavaScript, Rust, Go, C#
- Excluded: HTML, CSS, JSON, YAML, SQL

Output Summary

- Raw API results for 2007–2024 (optional)
- Final filtered and cleaned dataset in JSON format
- Language whitelist and blacklist separation
- Logging of remaining valid languages after processing

Notes

- GitHub API token is required and must be provided for authenticated requests
- Rate limits must be respected (5000 requests/hour for authenticated sessions)
- Only public repositories are queried

Author

Big Data Engineering
FH Technikum Wien, Summer Semester 2025
Manpreet Misson, Timothy Gregorian, Omar Sidi Mammar

Dependencies and Imports

This chunk brings in all the Python and third-party libraries required for HTTP requests, file I/O, JSON handling, sleeping between API calls, and data manipulation.

In [1]:
```
import os
import json
import requests
import pandas as pd
import time
from datetime import datetime
```

REST API Request Configuration

Sets the output directory and filename for the GitHub search results, specifies the search endpoint, and defines query parameters to fetch up to 500 of the most-starred repositories created in 2007.
We choose 2007 for this example because it only yields a single dataset, making the demonstration quick to execute.

In [2]:
```
GITHUB_TOKEN = "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"
HEADERS = {"Authorization": f"token {GITHUB_TOKEN}"}
restapi_dir = "./RestAPIData"
os.makedirs(restapi_dir, exist_ok=True)

output_file = os.path.join(restapi_dir, "Rest_API_Request2007.json")

url = "https://api.github.com/search/repositories"
params = {
    "q": "created:2007-01-01..2007-12-31",
    "sort": "stars",
    "order": "desc",
    "per_page": 500
}
```

Send API Request and Fetch Repo Metadata

Sends an authenticated request to GitHub's search API to retrieve metadata for top repositories created in 2007.
For each repo, a second request is made to get the language breakdown. Results are stored in a list and saved to disk.

In [3]:
```
response = requests.get(url, headers=HEADERS, params=params)
output_data = []

if response.status_code == 200:
    data = response.json()
    for repo in data["items"]:
```

```
        repo_info = {
            "full_name": repo["full_name"],
            "stargazers_count": repo["stargazers_count"],
            "created_at": repo["created_at"],
            "languages": {}
        }

        lang_url = f"https://api.github.com/repos/{repo['full_name']}/languages"
        lang_response = requests.get(lang_url, headers=HEADERS)
        if lang_response.status_code == 200:
            repo_info["languages"] = lang_response.json()

        output_data.append(repo_info)
        time.sleep(1)  # Rate-limiting between language requests

    with open(output_file, "w", encoding="utf-8") as f:
        json.dump(output_data, f, ensure_ascii=False, indent=4)
    print(f"Saved data to: {output_file}")
else:
    print(f"Request failed: {response.status_code}")
    print(response.text)
```

```
Saved data to: ./RestAPIData/Rest_API_Request2007.json
```

Load and Validate All API Files

Loads all previously saved JSON files from the output folder.

Each record is validated for required fields and de-duplicated by repo name.

In [4]:
```
valid_entries = []
seen = set()
for fname in sorted(os.listdir(restapi_dir)):
    if not (fname.startswith("Rest_API_Request") and fname.endswith(".json")):
        continue
    path = os.path.join(restapi_dir, fname)
    with open(path, "r", encoding="utf-8") as f:
        try:
            data = json.load(f)
        except json.JSONDecodeError:
            print(f"Skipping invalid JSON: {fname}")
            continue
    for entry in data:
        if not all([
            isinstance(entry, dict),
            entry.get("full_name"),
            entry.get("created_at"),
            isinstance(entry.get("stargazers_count"), (int, float)),
            isinstance(entry.get("languages"), dict)
        ]):
            continue
        if entry["full_name"] in seen:
            continue
        seen.add(entry["full_name"])
        valid_entries.append(entry)

print(f"Total combined valid entries: {len(valid_entries)}")
df = pd.DataFrame(valid_entries)
```

```
Total combined valid entries: 8501
```

Extract and Export All Unique Programming Languages

Parses the `languages` dictionary of each entry and saves the union of all keys across datasets.
This is helpful to inspect language diversity before applying filtering.

In [5]:
```
lang_set = set()
for langs in df["languages"]:
    lang_set.update(langs.keys())

lang_df = pd.DataFrame(sorted(lang_set), columns=["Language"])
lang_path = os.path.join(restapi_dir, "api_languages_unique.csv")
lang_df.to_csv(lang_path, index=False)
print(f"Saved language list to {lang_path}")
```

```
Saved language list to ./RestAPIData/api_languages_unique.csv
```

Define Whitelist and Create Blacklist

Compares collected languages with a canonical whitelist.
Languages not present in the whitelist are filtered out and saved separately for reference.

In [6]:
```
whitelist = {
    "Ada", "APL", "Assembly", "BASIC", "C", "C#", "C++", "COBOL", "Clojure", "Cr
    "D", "Dart", "Delphi", "Elixir", "Elm", "Erlang", "F#", "Fortran", "FreeBASI
    "Groovy", "Hack", "Haskell", "Java", "JavaScript", "Julia", "Kotlin", "Lisp"
    "Matlab", "Nim", "OCaml", "Objective-C", "Objective-C++", "Pascal", "Perl",
    "PowerShell", "Prolog", "Python", "R", "Raku", "Ruby", "Rust", "Scala", "Sch
    "Smalltalk", "Solidity", "Swift", "TypeScript", "VBA", "Visual Basic", "Zig"
}

blacklist_df = lang_df[~lang_df["Language"].isin(whitelist)]
blacklist_path = os.path.join(restapi_dir, "api_blacklist.csv")
blacklist_df.to_csv(blacklist_path, index=False)
print(f"Saved blacklist to {blacklist_path}")

blacklist = set(blacklist_df["Language"].str.lower().str.strip())
```

```
Saved blacklist to ./RestAPIData/api_blacklist.csv
```

Normalize Language Names in Language Dict

Corrects common misspellings or non-canonical variants to match the whitelist.

In [7]:
```
normalization_map = {
    "C++11": "C++", "Javascript": "JavaScript", "javascript": "JavaScript",
    "Typescript": "TypeScript", "typesSript": "TypeScript", "Golang": "Go",
    "MATLAB": "Matlab", "Matlab": "Matlab", "Perl 6": "Perl",
    "LISP": "Lisp", "Common Lisp": "Lisp", "Lisp": "Lisp", "Ocaml": "OCaml",
    "Delphi/Object Pascal": "Delphi", "Visual Basic (.Net)": "Visual Basic",
    "Visual Basic 6": "Visual Basic", "Visual Basic 6.0": "Visual Basic",
    "Visual Basic .NET": "Visual Basic", "VB.NET": "Visual Basic"
}

def normalize_lang_dict(lang_dict):
    return {
```

```
        normalization_map.get(k.strip(), k.strip()): v
        for k, v in lang_dict.items()
    }

df["languages"] = df["languages"].apply(normalize_lang_dict)
```

Apply Language Filter

Removes any language not on the whitelist by cleaning each dictionary of languages.

In [8]:
```
def filter_entry_langs(entry):
    return {
        lang: cnt for lang, cnt in entry["languages"].items()
        if lang.strip().lower() not in blacklist
    }

df["languages"] = df.apply(filter_entry_langs, axis=1)
df = df[df["languages"].map(bool)].reset_index(drop=True)
print(f"Records after language filter: {len(df)}")
```

```
Records after language filter: 7560
```

Filter Invalid Records (NA / Stars / Corrupt)

Ensures only valid data is retained by checking for empty fields and malformed values.

In [9]:
```
NA_PLACEHOLDERS = {"n/a", "na", "nan", "null", "none", ""}

def is_bad(val):
    if pd.isna(val):
        return True
    if isinstance(val, str) and val.strip().lower() in NA_PLACEHOLDERS:
        return True
    return False

cleaned_records = []
for rec in df.to_dict(orient="records"):
    if is_bad(rec["full_name"]) or is_bad(rec["created_at"]):
        continue
    stars = rec.get("stargazers_count")
    if not isinstance(stars, (int, float)) or stars < 0:
        continue
    cleaned_records.append(rec)

final_df = pd.DataFrame(cleaned_records)
print(f"Records after final NA check: {len(final_df)}")
```

```
Records after final NA check: 7560
```

Save Final Cleaned Output

Saves the cleaned and filtered REST API results to a JSON file for downstream analysis.

In [10]:
```
output_file = os.path.join(restapi_dir, "All_API_Requests_Cleaned.json")
with open(output_file, "w", encoding="utf-8") as f:
    json.dump(final_df.to_dict(orient="records"), f, ensure_ascii=False, indent=
print(f"Final cleaned data saved to: {output_file}")
```

```
Final cleaned data saved to: ./RestAPIData/All_API_Requests_Cleaned.json
```